

Бинарные файлы

1. Введение в бинарные файлы

Бинарный файл – это файл, содержащий данные в двоичном формате, который не является текстом. В отличие от текстовых файлов, бинарные файлы содержат последовательность байтов, которые интерпретируются по-особому в зависимости от типа данных.

Когда использовать бинарные файлы?

- Сохранение структурированных объектов (сериализация)
- Сохранение изображений, видео, аудио
- Компактное хранение данных с определённой структурой
- Защита данных от прямого редактирования
- Повышение скорости чтения/записи большого объёма данных

2. Различие между текстовыми и бинарными файлами

Текстовый файл

Содержит только текст в кодировке (UTF-8, ASCII и т.д.)

Человеко-читаемый формат

Каждая строка заканчивается специальным символом (перевод строки)

Можно открыть в любом текстовом редакторе

Бинарный файл

Содержит последовательность байтов

Не предназначен для прямого чтения человеком

Может хранить любые типы данных (числа, объекты, изображения)

Требуется специальная программа для интерпретации

Пример:

Число 255 в текстовом файле хранится как "255" (3 байта).

Число 255 в бинарном файле хранится как один байт: 11111111.

3. Класс File для работы с бинарными файлами

Класс File находится в пространстве имён System.IO и предоставляет статические методы для работы с файлами.

Основные методы для бинарных файлов:

Метод	Описание
File.WriteAllBytes(path, bytes)	Записывает массив байтов в файл
File.ReadAllBytes(path)	Читает весь файл как массив байтов
File.Exists(path)	Проверяет существование файла
File.Delete(path)	Удаляет файл
File.AppendAllBytes(path, bytes)	Добавляет байты в конец файла

Дополнительные классы для потоковой работы:

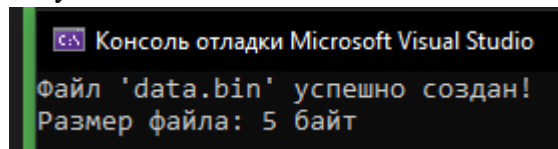
Класс	Описание
BinaryWriter	Запись примитивных типов в бинарный поток
BinaryReader	Чтение примитивных типов из бинарного потока
FileStream	Низкоуровневый доступ к файловой системе

4. Создание бинарного файла

Пример 1: Создание простого бинарного файла

```
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main()
7      {
8          // Путь к файлу
9          string filePath = "data.bin";
10
11         // Создаём массив байтов
12         byte[] data = { 10, 20, 30, 40, 50 };
13
14         // Записываем в файл
15         File.WriteAllBytes(filePath, data);
16
17         Console.WriteLine($"Файл '{filePath}' успешно создан!");
18         Console.WriteLine($"Размер файла: {new FileInfo(filePath).Length} байт");
19     }
20 }
```

Результат:



Консоль отладки Microsoft Visual Studio

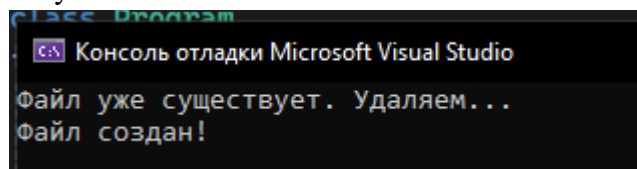
Файл 'data.bin' успешно создан!
Размер файла: 5 байт

FileInfo — это класс из пространства имён System.IO, который представляет информацию о конкретном файле на диске. Когда вы пишете `new FileInfo(путь)`, вы создаёте новый объект, который позволяет получать и манипулировать свойствами и методами этого файла.

Пример 2: Проверка существования файла перед созданием

```
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main()
7      {
8          string filePath = "data.bin";
9
10         // Проверяем, существует ли файл
11         if (File.Exists(filePath))
12         {
13             Console.WriteLine("Файл уже существует. Удаляем...");
14             File.Delete(filePath);
15         }
16
17         byte[] data = { 10, 20, 30, 40, 50 };
18         File.WriteAllBytes(filePath, data);
19
20         Console.WriteLine("Файл создан!");
21     }
22 }
```

Результат:

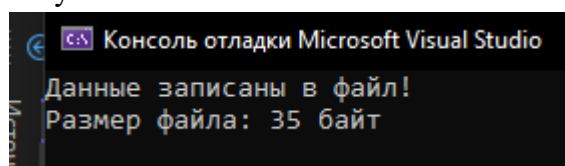


5. Запись данных в бинарный файл

Пример 1: Запись примитивных типов данных с использованием BinaryWriter

```
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main()
7      {
8          string filePath = "student.bin";
9
10         // Создаём файловый поток для записи
11         using (FileStream fs = new FileStream(filePath, FileMode.Create))
12         {
13             // Создаём BinaryWriter для удобной записи данных
14             using (BinaryWriter writer = new BinaryWriter(fs))
15             {
16                 // Записываем различные типы данных
17                 writer.Write("Иван Петров"); // строка (2 байта для длины + сама строка)
18                 writer.Write(20);           // int (4 байта)
19                 writer.Write(85.5);          // double (8 байт)
20                 writer.Write(true);          // bool (1 байт)
21             }
22         }
23
24         Console.WriteLine("Данные записаны в файл!");
25
26         // Проверяем размер файла
27         FileInfo fileInfo = new FileInfo(filePath);
28         Console.WriteLine($"Размер файла: {fileInfo.Length} байт");
29     }
30 }
```

Результат:



FileStream fs = new FileStream(filePath, FileMode.Create) – создаём новый объект FileStream. FileStream – это класс для низкоуровневого доступа к файлам. Он работает с потоком байтов. В скобках два параметра: filePath – путь к файлу, FileMode.Create – режим открытия. Create означает "создать новый файл". Если файл существует, он будет перезаписан

using (...) – конструкция using. Она гарантирует, что объект будет правильно закрыт и его ресурсы освобождены, даже если произойдёт ошибка. Это очень важно при работе с файлами, чтобы не "зависли" файлы

BinaryWriter writer = new BinaryWriter(fs) – создаём объект BinaryWriter. Это "переводчик", который помогает записывать разные типы данных (int, double, string и т.д.) в бинарный поток. Передаём ему FileStream fs, чтобы он знал, куда писать

writer.Write("Иван Петров") – метод Write класса BinaryWriter. Он записывает значение в файл. BinaryWriter автоматически "понимает", какой тип данных записывается (в этом случае string – строка). Для строк сначала записывается длина строки (2 байта), потом сама строка в кодировке UTF-8

writer.Write(20) – записываем целое число типа int. Занимает 4 байта в памяти

writer.Write(85.5) – записываем вещественное число типа double (число с дробной частью). Занимает 8 байт

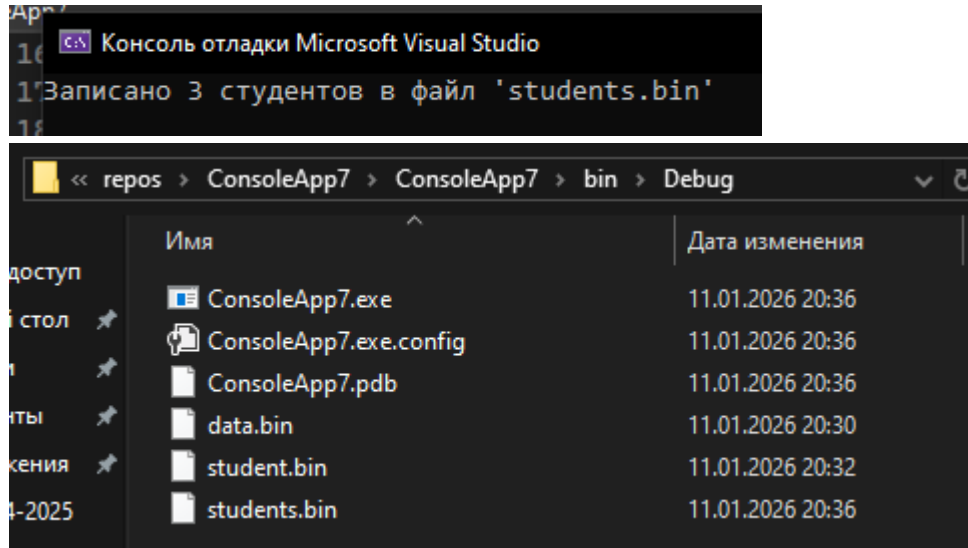
writer.Write(true) – записываем булево значение (true/false). Занимает 1 байт

FileInfo fileInfo = new FileInfo(filePath) – создаём объект FileInfo для получения информации о файле
fileInfo.Length – свойство, возвращающее размер файла в байтах.

Пример 2: Запись массива структур в бинарный файл

```
1  using System;
2  using System.IO;
3
4  // Определяем структуру для хранения данных
5  struct Student
6  {
7      public string Name;
8      public int Age;
9      public double GPA;
10 }
11
12 class Program
13 {
14     static void Main()
15     {
16         string filePath = "students.bin";
17
18         // Массив студентов
19         Student[] students = new Student[]
20         {
21             new Student { Name = "Иван", Age = 20, GPA = 4.5 },
22             new Student { Name = "Мария", Age = 21, GPA = 4.8 },
23             new Student { Name = "Петр", Age = 19, GPA = 4.2 }
24         };
25
26         // Записываем студентов в файл
27         using (FileStream fs = new FileStream(filePath, FileMode.Create))
28         {
29             using (BinaryWriter writer = new BinaryWriter(fs))
30             {
31                 // Сначала записываем количество студентов
32                 writer.Write(students.Length);
33
34                 // Затем записываем каждого студента
35                 foreach (var student in students)
36                 {
37                     writer.Write(student.Name);
38                     writer.Write(student.Age);
39                     writer.Write(student.GPA);
40                 }
41             }
42         }
43
44         Console.WriteLine($"Записано {students.Length} студентов в файл '{filePath}'");
45     }
46 }
```

Результат:

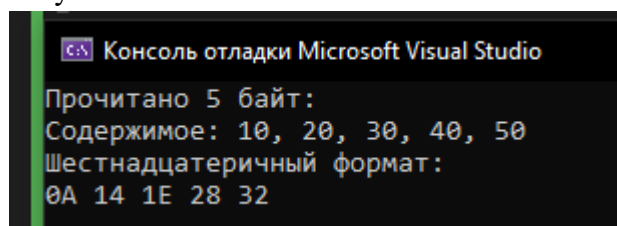


6. Чтение данных из бинарного файла

Пример 1: Чтение всех байтов за один раз

```
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main()
7      {
8          string filePath = "data.bin";
9
10         // Проверяем существование файла
11         if (!File.Exists(filePath))
12         {
13             Console.WriteLine("Файл не найден!");
14             return;
15         }
16
17         // Читаем весь файл как массив байтов
18         byte[] data = File.ReadAllBytes(filePath);
19
20         Console.WriteLine($"Прочитано {data.Length} байт:");
21         Console.WriteLine("Содержимое: " + string.Join(", ", data));
22
23         // Выводим в шестнадцатеричном формате
24         Console.WriteLine("Шестнадцатеричный формат:");
25         foreach (var b in data)
26         {
27             Console.Write($"{b:X2} "); // X2 = шестнадцатеричный формат с двумя цифрами
28         }
29         Console.WriteLine();
30     }
31 }
```

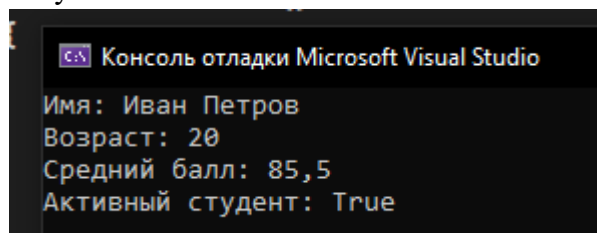
Результат:



Пример 2: Чтение примитивных типов с использованием BinaryReader

```
1  using System;
2  using System.IO;
3
4  class Program
5  {
6      static void Main()
7      {
8          string filePath = "student.bin";
9
10         // Проверяем существование файла
11         if (!File.Exists(filePath))
12         {
13             Console.WriteLine("Файл не найден!");
14             return;
15         }
16
17         // Открываем файл для чтения
18         using (FileStream fs = new FileStream(filePath, FileMode.Open))
19         {
20             using (BinaryReader reader = new BinaryReader(fs))
21             {
22                 // Читаем данные в том же порядке, в котором они записаны
23                 string name = reader.ReadString();
24                 int age = reader.ReadInt32();
25                 double gpa = reader.ReadDouble();
26                 bool isActive = reader.ReadBoolean();
27
28                 Console.WriteLine($"Имя: {name}");
29                 Console.WriteLine($"Возраст: {age}");
30                 Console.WriteLine($"Средний балл: {gpa}");
31                 Console.WriteLine($"Активный студент: {isActive}");
32             }
33         }
34     }
35 }
```

Результат:



Консоль отладки Microsoft Visual Studio

```
Имя: Иван Петров
Возраст: 20
Средний балл: 85,5
Активный студент: True
```

Пример 3: Чтение массива структур из бинарного файла

```
1  using System;
2  using System.IO;
3
4  struct Student
5  {
6      public string Name;
7      public int Age;
8      public double GPA;
9
10     public override string ToString()
11     {
12         return $"Студент: {Name}, Возраст: {Age}, Средний балл: {GPA}";
13     }
14 }
15
16 class Program
17 {
18     static void Main()
19     {
20         string filePath = "students.bin";
21
22         if (!File.Exists(filePath))
23         {
24             Console.WriteLine("Файл не найден!");
25             return;
26         }
27
28         using (FileStream fs = new FileStream(filePath, FileMode.Open))
29         {
30             using (BinaryReader reader = new BinaryReader(fs))
31             {
32                 // Читаем количество студентов
33                 int count = reader.ReadInt32();
34                 Console.WriteLine($"Всего студентов: {count}\n");
35
36                 // Читаем каждого студента
37                 for (int i = 0; i < count; i++)
38                 {
39                     Student student = new Student
40                     {
41                         Name = reader.ReadString(),
42                         Age = reader.ReadInt32(),
43                         GPA = reader.ReadDouble()
44                     };
45
46                     Console.WriteLine(student);
47                 }
48             }
49         }
50     }
51 }
```

Результат:

```
m.
c:\> Консоль отладки Microsoft Visual Studio
Всего студентов: 3

Студент: Иван, Возраст: 20, Средний балл: 4,5
Студент: Мария, Возраст: 21, Средний балл: 4,8
Студент: Петр, Возраст: 19, Средний балл: 4,2
```